ARCHON — Episode 1: Scalars Vectors and Vector Spaces

March 23, 2026 • Phase: 1 • linear_algebra

Linear Algebra for AI/ML: Episode 1

Welcome to Your Linear Algebra Journey!

Welcome to the first episode of your linear algebra foundations series! Over the coming weeks, we'll build your mathematical foundation from the ground up, starting with today's topic on vectors and vector spaces.

You're about to discover that linear algebra isn't just about manipulating numbers—it's a powerful language for describing relationships, transformations, and patterns that forms the backbone of modern machine learning. Whether you're implementing neural networks, understanding dimensionality reduction, or working with recommendation systems, the concepts we'll cover today will appear everywhere.

Don't worry if you've never taken a formal linear algebra course. We'll start from absolute first principles, assuming only high school math knowledge. By the end of today's session, you'll understand vectors not just as "lists of numbers" but as fundamental mathematical objects with rich geometric meaning.

Today's Topic: Scalars, Vectors, and Vector Spaces

Today we're laying the foundation for everything that follows. Vectors are to linear algebra what variables are to programming—they're the basic building blocks we'll use to construct more complex ideas. But unlike simple variables that hold single values, vectors can represent multiple dimensions of information simultaneously.

Why Vectors Matter in AI/ML

In machine learning, almost everything is a vector: - A grayscale image? That's a vector where each component represents a pixel intensity - A word in natural language processing? Often represented as a high-dimensional vector - The weights in a neural network layer? Vectors - The features describing a data point? Also a vector

Understanding vectors deeply means understanding how to manipulate, transform, and reason about data in machine learning. Today's concepts will directly apply when you: - Implement gradient descent (vectors represent parameter updates) - Work with embeddings (words, images, or other data mapped to vector spaces) - Understand neural network architectures (layers transform vectors) - Perform dimensionality reduction (projecting high-dimensional vectors to lower dimensions)

Our Learning Path Today

We'll start with the distinction between scalars and vectors, then build up your geometric intuition for what vectors represent. From there, we'll explore operations on vectors, the concept of linear

independence, and finally formalize everything with vector spaces. Along the way, you'll write Python code to make these abstract concepts concrete.

Core Concepts

Scalars: The Simple Numbers

Let's start with the simplest mathematical object: the scalar. A scalar is just a single real number—nothing more, nothing less.

Notation: We typically write scalars in italic font: a, b, c, α, β

Examples of scalars: - Temperature: $t = 25.5$ (degrees Celsius) - Learning rate in gradient descent: $\alpha = 0.01$ - A neural network weight: $w = -0.73$

Scalars represent magnitude without direction. When we say "5 kilometers," we're using a scalar—it tells us how much but not which way.

Vectors: Collections with Structure

Now for the star of today's show: vectors.

Intuition First: Imagine you're giving directions to a friend. You could say "walk 3 blocks," but that's incomplete—which direction? A better instruction would be "walk 3 blocks north and 2 blocks east." This pair of numbers (3, 2) with their associated directions forms a vector.

Formal Definition: A vector is an ordered list of numbers (called components or elements). In n-dimensional space, a vector has n components.

Notation: We write vectors in bold lowercase letters: $\mathbf{v}$, $\mathbf{u}$, $\mathbf{x}$

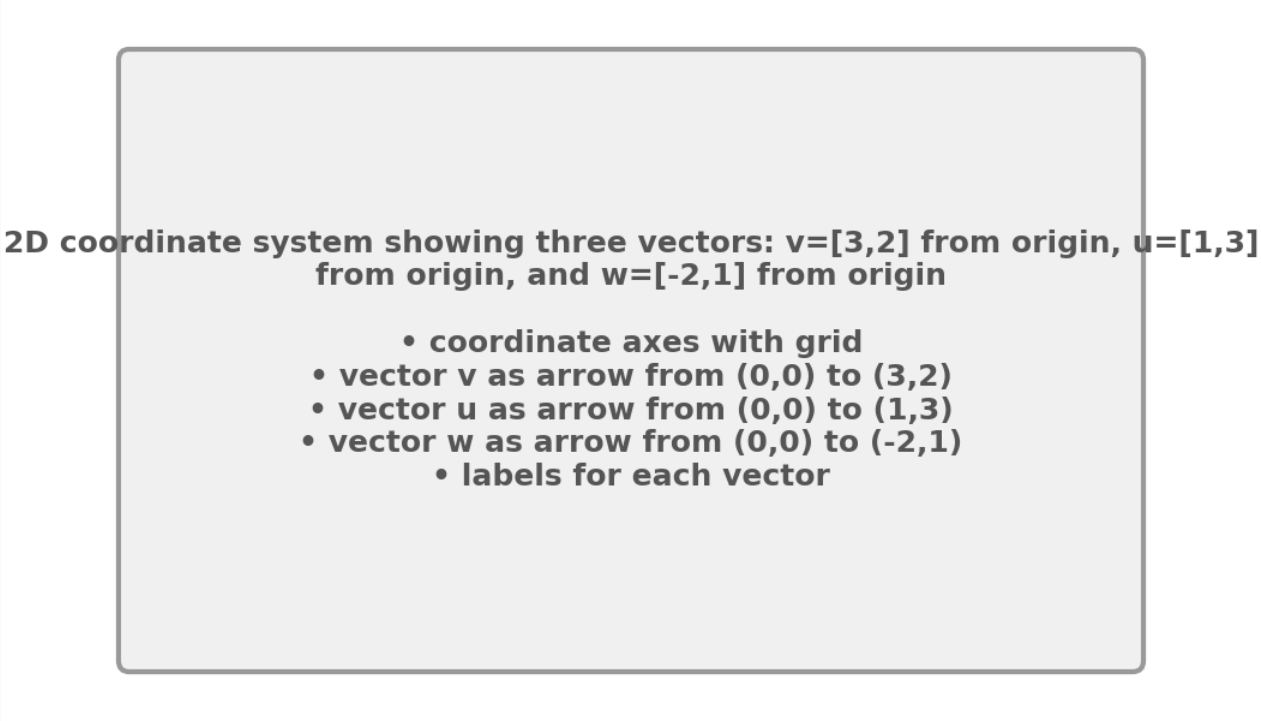
A vector in $\mathbb{R}^2$ (2D space): $\mathbf{v} = [v_1 \ v_2] = [3 \ 2]$

A vector in $\mathbb{R}^3$ (3D space): $\mathbf{u} = [u_1 \ u_2 \ u_3] = [1 \ -2 \ 5]$

Column vs Row Vectors: By convention, we typically write vectors as columns. A row vector would be written as $\mathbf{v}^T = [v_1, v_2]$ where T denotes transpose.

Geometric Interpretation of Vectors

Vectors have a beautiful geometric interpretation as arrows in space.



Key geometric properties:

1. **Magnitude (Length):** The length of the arrow, calculated using the Pythagorean theorem:

$$||\mathbf{v}|| = \sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}$$

For $\mathbf{v} = [3\ 4]$, the magnitude is $||\mathbf{v}|| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$

1. **Direction:** Where the arrow points. Two vectors with the same direction but different magnitudes are parallel.

2. **Position vs Free Vectors:**

3. Position vector: Fixed at the origin, points to a specific location
4. Free vector: Can be moved anywhere while maintaining magnitude and direction

Let's implement this in Python:

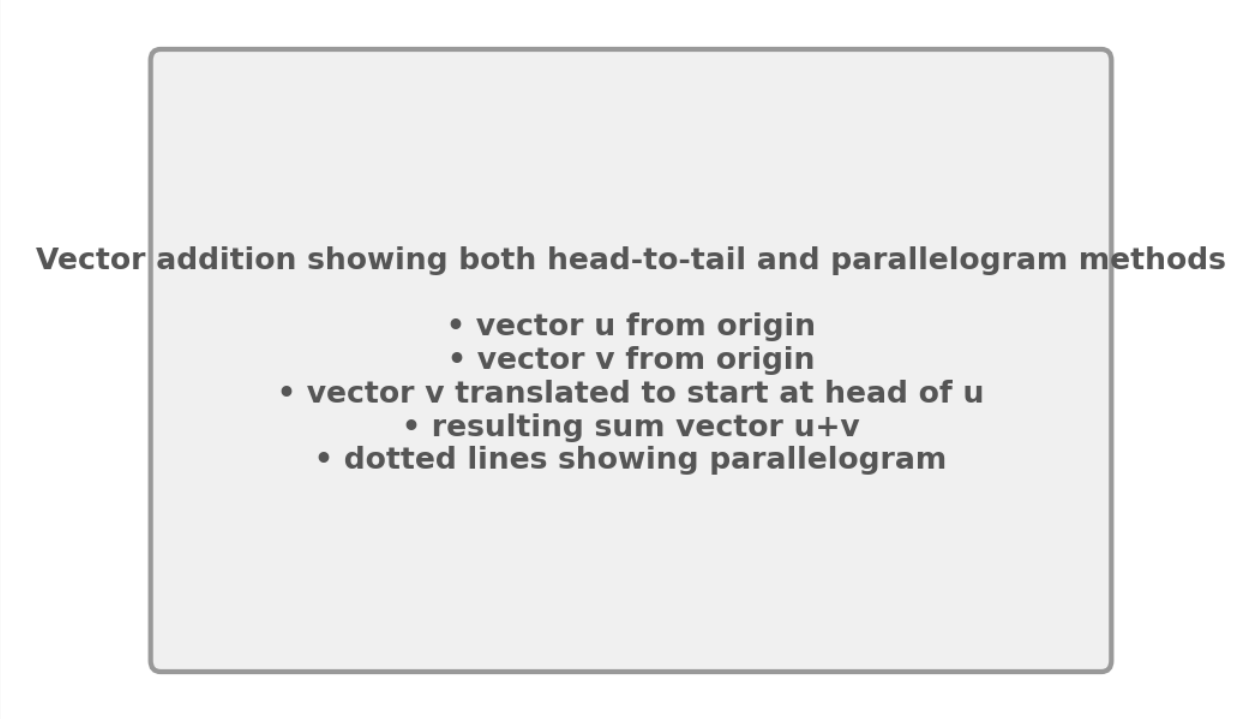
```
import numpy as np
import matplotlib.pyplot as plt

# Create vectors as NumPy arrays
v = np.array([3, 2])  # Creates a 1D array representing our vector
u = np.array([1, 3])
w = np.array([-2, 1])

# Calculate magnitude (length) of vector v
magnitude_v = np.linalg.norm(v)  # np.linalg.norm computes the Euclidean norm (length)
print(f"Magnitude of v: {magnitude_v:.2f}")  # Should print 3.61

# Visualize vectors
plt.figure(figsize=(8, 8))
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

# Plot vectors as arrows from origin
# quiver(x_start, y_start, x_component, y_component)
plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.005)
plt.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.005)
plt.quiver(0, 0, w[0], w[1], angles='xy', scale_units='xy', scale=1, color='green', width=0.005)

# Add labels
plt.text(v[0]+0.1, v[1]+0.1, 'v = [3, 2]', fontsize=12)
plt.text(u[0]+0.1, u[1]+0.1, 'u = [1, 3]', fontsize=12)
plt.text(w[0]-0.5, w[1]+0.1, 'w = [-2, 1]', fontsize=12)

plt.xlim(-3, 4)
plt.ylim(-1, 4)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Vectors in 2D Space')
plt.show()
```

Vector Addition: Combining Vectors

Vector addition combines two vectors to produce a third vector. There are two ways to think about this:

Geometric Interpretation (Parallelogram Rule): When adding vectors $\mathbf{u}$ and $\mathbf{v}$: 1. Place the tail of $\mathbf{v}$ at the head of $\mathbf{u}$ (head-to-tail method) 2. The sum $\mathbf{u}+\mathbf{v}$ is the vector from the tail of $\mathbf{u}$ to the head of $\mathbf{v}$

Alternatively: 1. Draw both vectors from the same starting point 2. Complete the parallelogram 3. The diagonal is $\mathbf{u}+\mathbf{v}$



Algebraic Definition: For vectors $\mathbf{u} = [u_1\ u_2\ \ldots\ u_n]$ and $\mathbf{v} = [v_1\ v_2\ \ldots\ v_n]$:

$$\mathbf{u} + \mathbf{v} = [u_1 + v_1\ u_2 + v_2\ \ldots\ u_n + v_n]$$

Properties of Vector Addition: 1. **Commutative:** $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$ 2. **Associative:** $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ 3. **Identity:** There exists a zero vector $\mathbf{0}$ such that $\mathbf{v} + \mathbf{0} = \mathbf{v}$ 4. **Inverse:** For every vector $\mathbf{v}$, there exists $-\mathbf{v}$ such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$

Let's implement and visualize vector addition:

```
def visualize_vector_addition(u, v):
    """Visualize vector addition using both methods"""
    # Calculate the sum
    w = u + v  # NumPy handles component-wise addition automatically

    plt.figure(figsize=(10, 5))

    # Subplot 1: Head-to-tail method
    plt.subplot(1, 2, 1)
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)

    # Draw u from origin
    plt.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1,
               color='red', width=0.005, label='u')
    # Draw v from head of u
    plt.quiver(u[0], u[1], v[0], v[1], angles='xy', scale_units='xy', scale=1,
               color='blue', width=0.005, label='v')
    # Draw sum from origin
    plt.quiver(0, 0, w[0], w[1], angles='xy', scale_units='xy', scale=1,
               color='green', width=0.005, label='u + v')

    plt.title('Head-to-Tail Method')
    plt.legend()
    plt.axis('equal')

    # Subplot 2: Parallelogram method
    plt.subplot(1, 2, 2)
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)

    # Draw both vectors from origin
    plt.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1,
               color='red', width=0.005, label='u')
    plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1,
               color='blue', width=0.005, label='v')
    # Draw sum
    plt.quiver(0, 0, w[0], w[1], angles='xy', scale_units='xy', scale=1,
               color='green', width=0.005, label='u + v')

    # Draw parallelogram with dashed lines
    plt.plot([u[0], w[0]], [u[1], w[1]], 'k--', alpha=0.5)  # From u to u+v
    plt.plot([v[0], w[0]], [v[1], w[1]], 'k--', alpha=0.5)  # From v to u+v

    plt.title('Parallelogram Method')
    plt.legend()
```

```
    plt.axis('equal')

    plt.tight_layout()
    plt.show()

    # Print the calculation
    print(f"u = {u}")
    print(f"v = {v}")
    print(f"u + v = {w}")

# Example usage
u = np.array([3, 1])
v = np.array([1, 2])
visualize_vector_addition(u, v)
```

Scalar Multiplication: Stretching and Shrinking

Scalar multiplication scales a vector by a constant factor, changing its magnitude while preserving (or reversing) its direction.

Algebraic Definition: For scalar c and vector $\mathbf{v} = [v_1\ v_2\ \ldots\ v_n]$:

$$c\mathbf{v} = [cv_1\ cv_2\ \ldots\ cv_n]$$

Geometric Interpretation: - If $c\>1$: The vector stretches (gets longer) - If $0\<c\<1$: The vector shrinks (gets shorter) - If $c\<0$: The vector reverses direction - If $c = 0$: The result is the zero vector - If $c = -1$: We get the inverse vector

Properties of Scalar Multiplication: 1. **Distributive over vector addition:** $c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$ 2. **Distributive over scalar addition:** $(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$ 3. **Associative:** $(cd)\mathbf{v} = c(d\mathbf{v})$ 4. **Identity:** $1\mathbf{v} = \mathbf{v}$

```
def visualize_scalar_multiplication(v, scalars):
    """Visualize scalar multiplication with multiple scalars"""
    plt.figure(figsize=(10, 8))
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)

    # Color map for different scalars
    colors = plt.cm.coolwarm(np.linspace(0, 1, len(scalars)))

    for i, c in enumerate(scalars):
        scaled_v = c * v  # NumPy handles scalar multiplication naturally

        # Plot the scaled vector
        plt.quiver(0, 0, scaled_v[0], scaled_v[1],
                   angles='xy', scale_units='xy', scale=1,
                   color=colors[i], width=0.005,
                   label=f'{c}v')

        # Add text label at the head of the vector
        plt.text(scaled_v[0] + 0.1, scaled_v[1] + 0.1, f'{c}v', fontsize=10)

    # Plot original vector in black for reference
    plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1,
               color='black', width=0.007, label='v (original)')

    plt.title('Scalar Multiplication Effects')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.axis('equal')
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.tight_layout()
    plt.show()

    # Print magnitudes
    print(f"Original vector v = {v}")
    print(f"Magnitude of v: {np.linalg.norm(v):.2f}")
    print("\nScaled vectors:")
    for c in scalars:
        scaled_v = c * v
        print(f"{c}v = {scaled_v}, magnitude = {np.linalg.norm(scaled_v):.2f}")

# Example
v = np.array([2, 1])
scalars = [2, 0.5, -1, -0.5, 0]
visualize_scalar_multiplication(v, scalars)
```

Linear Combinations and Linear Independence

A **linear combination** of vectors is formed by multiplying each vector by a scalar and adding the results:

$$\mathbf{w} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \ldots + c_k\mathbf{v}_k$$

This concept is fundamental because: - Any vector in a space can be expressed as a linear combination of basis vectors - Machine learning models often compute linear combinations (e.g., in neural network layers)

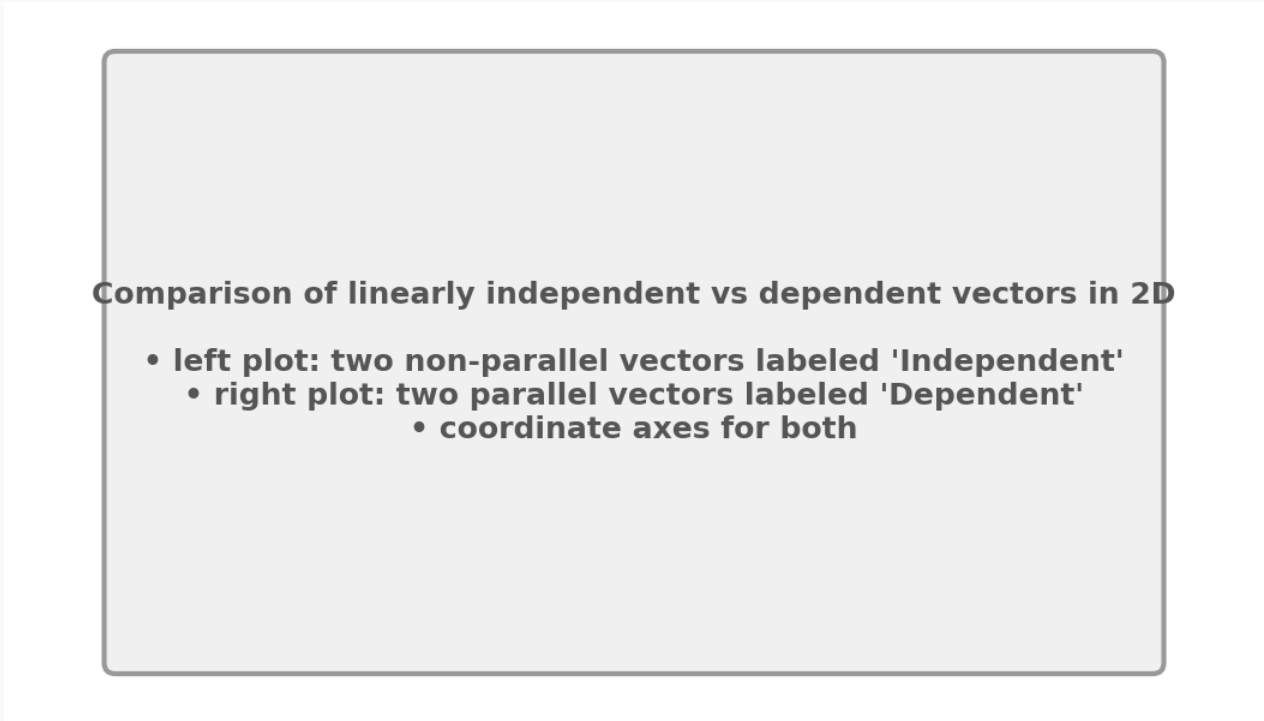
Linear Independence: A set of vectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k$ is **linearly independent** if the only solution to:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \ldots + c_k\mathbf{v}_k = \mathbf{0}$$

is $c_1 = c_2 = \ldots = c_k = 0$.

Intuition: Vectors are linearly independent if no vector in the set can be written as a linear combination of the others. They point in fundamentally different directions.

Geometric Interpretation: - In 2D: Two vectors are linearly independent if they're not parallel (not on the same line) - In 3D: Three vectors are linearly independent if they don't all lie in the same plane



```
def check_linear_independence_2d(v1, v2):
    """
    Check if two 2D vectors are linearly independent.
    Two vectors are dependent if one is a scalar multiple of the other.
    """
    # Method 1: Check if vectors are parallel (cross product = 0)
    cross_product = v1[0] * v2[1] - v1[1] * v2[0]

    # Method 2: Check determinant of matrix formed by vectors
    matrix = np.column_stack([v1, v2])  # Create matrix with v1 and v2 as columns
    determinant = np.linalg.det(matrix)

    # Vectors are independent if determinant is non-zero
    is_independent = not np.isclose(determinant, 0)

    return is_independent, determinant, cross_product

# Examples
v1 = np.array([1, 2])
v2 = np.array([2, 4])  # This is 2 * v1, so dependent
v3 = np.array([1, 3])  # Not a multiple of v1, so independent

print("Testing v1 = [1, 2] and v2 = [2, 4]:")
independent, det, cross = check_linear_independence_2d(v1, v2)
print(f"  Independent: {independent}")
print(f"  Determinant: {det}")
print(f"  Cross product: {cross}")

print("\nTesting v1 = [1, 2] and v3 = [1, 3]:")
independent, det, cross = check_linear_independence_2d(v1, v3)
print(f"  Independent: {independent}")
print(f"  Determinant: {det}")
print(f"  Cross product: {cross}")

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Dependent vectors
ax1.grid(True, alpha=0.3)
ax1.axhline(y=0, color='k', linewidth=0.5)
ax1.axvline(x=0, color='k', linewidth=0.5)
ax1.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.005)
ax1.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.005)
ax1.text(v1[0]+0.1, v1[1]+0.1, 'v1', fontsize=12)
ax1.text(v2[0]+0.1, v2[1]+0.1, 'v2 = 2v1', fontsize=12)
ax1.set_title('Linearly Dependent Vectors')
ax1.set_xlim(-1, 3)
ax1.set_ylim(-1, 5)
```

```
# Independent vectors
ax2.grid(True, alpha=0.3)
ax2.axhline(y=0, color='k', linewidth=0.5)
ax2.axvline(x=0, color='k', linewidth=0.5)
ax2.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.005)
ax2.quiver(0, 0, v3[0], v3[1], angles='xy', scale_units='xy', scale=1, color='green', width=0.005)
ax2.text(v1[0]+0.1, v1[1]+0.1, 'v1', fontsize=12)
ax2.text(v3[0]+0.1, v3[1]+0.1, 'v3', fontsize=12)
ax2.set_title('Linearly Independent Vectors')
ax2.set_xlim(-1, 3)
ax2.set_ylim(-1, 5)

plt.tight_layout()
plt.show()
```

Basis Vectors and Span

Basis Vectors: A basis for a vector space is a set of linearly independent vectors that span the entire space. Every vector in the space can be uniquely expressed as a linear combination of basis vectors.

Standard Basis: In $\mathbb{R}^n$, the standard basis consists of vectors with a 1 in one position and 0s elsewhere:

For $\mathbb{R}^2$: $\mathbf{e}_1 = [1\ 0]$, $\mathbf{e}_2 = [0\ 1]$

For $\mathbb{R}^3$: $\mathbf{e}_1 = [1\ 0\ 0]$, $\mathbf{e}_2 = [0\ 1\ 0]$, $\mathbf{e}_3 = [0\ 0\ 1]$

Any vector can be written using the standard basis:

$$\mathbf{v} = [3\ 5] = 3\mathbf{e}_1 + 5\mathbf{e}_2 = 3[1\ 0] + 5[0\ 1]$$

Span: The span of a set of vectors is the set of all possible linear combinations of those vectors.

$$\text{span}\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \ldots + c_k\mathbf{v}_k : c_i \in \mathbb{R}$$

Geometric interpretation: - Span of one non-zero vector: a line through the origin - Span of two linearly independent vectors in $\mathbb{R}^3$: a plane through the origin - Span of three linearly independent vectors in $\mathbb{R}^3$: all of $\mathbb{R}^3$

```
def visualize_span_2d(v1, v2=None):
    """Visualize the span of one or two vectors in 2D"""
    fig, ax = plt.subplots(figsize=(8, 8))
    ax.grid(True, alpha=0.3)
    ax.axhline(y=0, color='k', linewidth=0.5)
    ax.axvline(x=0, color='k', linewidth=0.5)

    if v2 is None or np.allclose(np.cross(v1, v2), 0):
        # Span of one vector (or two dependent vectors) is a line
        t = np.linspace(-3, 3, 100)
        line_x = t * v1[0]
        line_y = t * v1[1]
        ax.plot(line_x, line_y, 'g-', alpha=0.3, linewidth=3, label='span (line)')
        ax.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1,
                  color='red', width=0.005, label='v1')
        title = 'Span of One Vector (Line)'
    else:
        # Span of two independent vectors is the entire plane
        # Show this by plotting many linear combinations
        coeffs = np.linspace(-2, 2, 20)
        for c1 in coeffs[::4]:  # Plot every 4th line for clarity
            for c2 in coeffs[::4]:
                point = c1 * v1 + c2 * v2
                ax.plot(point[0], point[1], 'go', alpha=0.3, markersize=3)

        # Highlight the basis vectors
        ax.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1,
                  color='red', width=0.005, label='v1', zorder=5)
        ax.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1,
                  color='blue', width=0.005, label='v2', zorder=5)

        # Show a few example linear combinations
        examples = [(1, 1), (2, 1), (1, 2), (-1, 1)]
        for c1, c2 in examples:
            combo = c1 * v1 + c2 * v2
            ax.quiver(0, 0, combo[0], combo[1], angles='xy', scale_units='xy',
                      scale=1, color='purple', width=0.003, alpha=0.7)
            ax.text(combo[0]+0.1, combo[1]+0.1, f'{c1}v1+{c2}v2', fontsize=8)

        title = 'Span of Two Independent Vectors (Entire Plane)'

    ax.set_xlim(-5, 5)
    ax.set_ylim(-5, 5)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_title(title)
    ax.legend()
    ax.set_aspect('equal')
```

```
    plt.show()

# Examples
print("Span of a single vector:")
visualize_span_2d(np.array([2, 1]))

print("\nSpan of two independent vectors:")
visualize_span_2d(np.array([2, 1]), np.array([1, 2]))
```

Vector Spaces: The Formal Framework

A **vector space** is a set V of vectors together with two operations (vector addition and scalar multiplication) that satisfy eight axioms. This abstract definition captures the essential properties that make vectors useful.

The Eight Vector Space Axioms:

For all vectors $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all scalars $c, d \in \mathbb{R}$:

1. **Closure under addition:** $\mathbf{u} + \mathbf{v} \in V$
2. **Closure under scalar multiplication:** $c\mathbf{v} \in V$
3. **Associativity of addition:** $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
4. **Commutativity of addition:** $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
5. **Additive identity:** There exists $\mathbf{0} \in V$ such that $\mathbf{v} + \mathbf{0} = \mathbf{v}$
6. **Additive inverse:** For each $\mathbf{v} \in V$, there exists $-\mathbf{v} \in V$ such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$
7. **Multiplicative identity:** $1\mathbf{v} = \mathbf{v}$
8. **Distributivity:**
9. $c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$
10. $(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$
11. $c(d\mathbf{v}) = (cd)\mathbf{v}$

Common Examples of Vector Spaces: - $\mathbb{R}^n$: All n-dimensional real vectors - Polynomials of degree at most n - $m \times n$ matrices - Continuous functions on an interval

Subspaces: Vector Spaces Within Vector Spaces

A **subspace** is a subset of a vector space that is itself a vector space under the same operations.

Subspace Test: A subset W of vector space V is a subspace if and only if: 1. The zero vector is in W: $\mathbf{0} \in W$ 2. W is closed under addition: If $\mathbf{u}, \mathbf{v} \in W$, then $\mathbf{u} + \mathbf{v} \in W$ 3. W is closed under scalar multiplication: If $\mathbf{v} \in W$ and $c \in \mathbb{R}$, then $c\mathbf{v} \in W$

Examples of Subspaces: - Any line through the origin in $\mathbb{R}^2$ or $\mathbb{R}^3$ - Any plane through the origin in $\mathbb{R}^3$ - The span of any set of vectors (always includes zero)

Non-examples: - A line not passing through the origin (doesn't contain zero vector) - A circle (not closed under scalar multiplication)

```
def verify_subspace_2d(vectors, name):
    """
    Verify if a set of 2D points forms a subspace.
    For demonstration, we'll check a finite set of points,
    but remember that subspaces are infinite sets!
    """
    print(f"\nChecking if {name} could represent a subspace:")

    # Convert to numpy array
    vectors = np.array(vectors)

    # Test 1: Contains zero vector?
    contains_zero = any(np.allclose(v, [0, 0]) for v in vectors)
    print(f"  Contains zero vector: {contains_zero}")

    # Test 2: Closed under addition? (check a few examples)
    addition_closed = True
    for i in range(min(3, len(vectors))):
        for j in range(min(3, len(vectors))):
            sum_vec = vectors[i] + vectors[j]
            # In a real subspace, the sum should follow the pattern
            print(f"  v{i} + v{j} = {vectors[i]} + {vectors[j]} = {sum_vec}")

    # Test 3: Closed under scalar multiplication? (check a few examples)
    for i in range(min(2, len(vectors))):
        for scalar in [2, -1, 0.5]:
            scaled = scalar * vectors[i]
            print(f"  {scalar} * v{i} = {scalar} * {vectors[i]} = {scaled}")

    return contains_zero

# Example: Points on a line through origin (subspace)
t = np.linspace(-2, 2, 5)
line_vectors = [[t_i, 2*t_i] for t_i in t]  # Points on line y = 2x
verify_subspace_2d(line_vectors, "Line through origin: y = 2x")

# Example: Points on a line NOT through origin (not a subspace)
line_offset = [[t_i, 2*t_i + 1] for t_i in t]  # Points on line y = 2x + 1
verify_subspace_2d(line_offset, "Line with offset: y = 2x + 1")
```

NumPy Operations: Bringing It All Together

Let's consolidate our learning with a comprehensive NumPy example that demonstrates all the vector operations we've covered:

```
import numpy as np

# Creating vectors in NumPy
print("=== Creating Vectors ===")

# Method 1: From a list
v1 = np.array([1, 2, 3])
print(f"v1 from list: {v1}")
print(f"Shape of v1: {v1.shape}")  # (3,) means 1D array with 3 elements
print(f"Data type: {v1.dtype}")

# Method 2: Using numpy functions
v2 = np.zeros(3)  # Vector of zeros
v3 = np.ones(3)   # Vector of ones
v4 = np.arange(0, 10, 2)  # [0, 2, 4, 6, 8]
v5 = np.linspace(0, 1, 5)  # 5 evenly spaced points from 0 to 1

print(f"Zero vector: {v2}")
print(f"Ones vector: {v3}")
print(f"Range vector: {v4}")
print(f"Linspace vector: {v5}")

# Column vectors (2D arrays with shape (n, 1))
col_vector = np.array([[1], [2], [3]])  # Note the extra brackets
print(f"\nColumn vector shape: {col_vector.shape}")  # (3, 1)

# Vector operations
print("\n=== Vector Operations ===")
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Addition and subtraction
print(f"a = {a}")
print(f"b = {b}")
print(f"a + b = {a + b}")  # Element-wise addition
print(f"a - b = {a - b}")  # Element-wise subtraction

# Scalar multiplication
print(f"3 * a = {3 * a}")
print(f"a / 2 = {a / 2}")

# Linear combination
c1, c2 = 2, -1
result = c1 * a + c2 * b
print(f"{c1}*a + {c2}*b = {result}")

# Magnitude (norm)
print("\n=== Vector Magnitude ===")
```

```
v = np.array([3, 4])
magnitude = np.linalg.norm(v)  # Euclidean norm (L2 norm)
print(f"v = {v}")
print(f"||v|| = {magnitude}")  # Should be 5 (3-4-5 triangle)

# Other norms
print(f"L1 norm: {np.linalg.norm(v, 1)}")  # Sum of absolute values
print(f"L∞ norm: {np.linalg.norm(v, np.inf)}")  # Maximum absolute value

# Creating unit vectors (vectors with magnitude 1)
print("\n=== Unit Vectors ===")
v = np.array([3, 4])
unit_v = v / np.linalg.norm(v)  # Normalize the vector
print(f"Original vector: {v}")
print(f"Unit vector: {unit_v}")
print(f"Magnitude of unit vector: {np.linalg.norm(unit_v)}")

# Working with matrices (for linear independence)
print("\n=== Linear Independence Check ===")
# Create matrix with vectors as columns
v1 = np.array([1, 0])
v2 = np.array([0, 1])
v3 = np.array([1, 1])

# Check if v1 and v2 are linearly independent
A = np.column_stack([v1, v2])  # Matrix with v1, v2 as columns
print(f"Matrix A with v1, v2 as columns:\n{A}")
print(f"Determinant: {np.linalg.det(A)}")  # Non-zero means independent
print(f"Rank: {np.linalg.matrix_rank(A)}")  # Rank = 2 means both vectors are independent

# Check if v1, v2, v3 are linearly independent in R^2
# (Spoiler: they can't be, as we can have at most 2 independent vectors in R^2)
B = np.column_stack([v1, v2, v3])
print(f"\nMatrix B with v1, v2, v3 as columns:\n{B}")
print(f"Rank: {np.linalg.matrix_rank(B)}")  # Rank = 2, so only 2 are independent

# Advanced: Projection (preview for next lesson)
print("\n=== Preview: Projection ===")
# Project vector a onto vector b
a = np.array([3, 4])
b = np.array([1, 0])  # Project onto x-axis
projection = (np.dot(a, b) / np.dot(b, b)) * b
print(f"Projecting {a} onto {b}: {projection}")

# Reshaping vectors
print("\n=== Reshaping ===")
v = np.array([1, 2, 3, 4, 5, 6])
print(f"Original vector: {v}, shape: {v.shape}")
```

```
# Reshape to 2x3 matrix
matrix = v.reshape(2, 3)
print(f"Reshaped to 2x3:\n{matrix}")

# Reshape to column vector
col = v.reshape(-1, 1)  # -1 means "infer this dimension"
print(f"As column vector: shape {col.shape}")
```

Worked Examples

Example 1: Vector Operations Workout

Problem: Given vectors $\mathbf{u} = [1\ 2]$ and $\mathbf{v} = [3\ -1]$, compute: a) $\mathbf{u} + \mathbf{v}$ b) $2\mathbf{u}$ c) $\mathbf{u} - \mathbf{v}$ d) $3\mathbf{u} + 2\mathbf{v}$ e) The magnitude of each result

Solution:

Let's solve this step by step both algebraically and with code:

```
# Define the vectors
u = np.array([1, 2])
v = np.array([3, -1])

print("Given vectors:")
print(f"u = {u}")
print(f"v = {v}")

# Part a: u + v
u_plus_v = u + v
print(f"\na) u + v = {u} + {v} = {u_plus_v}")
print(f"   Algebraically: [1+3, 2+(-1)] = [4, 1]")
print(f"   Magnitude: ||u + v|| = {np.linalg.norm(u_plus_v):.2f}")

# Part b: 2u
two_u = 2 * u
print(f"\nb) 2u = 2 * {u} = {two_u}")
print(f"   Algebraically: [2*1, 2*2] = [2, 4]")
print(f"   Magnitude: ||2u|| = {np.linalg.norm(two_u):.2f}")

# Part c: u - v
u_minus_v = u - v
print(f"\nc) u - v = {u} - {v} = {u_minus_v}")
print(f"   Algebraically: [1-3, 2-(-1)] = [-2, 3]")
print(f"   Magnitude: ||u - v|| = {np.linalg.norm(u_minus_v):.2f}")

# Part d: 3u + 2v
result_d = 3 * u + 2 * v
print(f"\nd) 3u + 2v = 3 * {u} + 2 * {v} = {result_d}")
print(f"   Step 1: 3u = [3*1, 3*2] = [3, 6]")
print(f"   Step 2: 2v = [2*3, 2*(-1)] = [6, -2]")
print(f"   Step 3: 3u + 2v = [3+6, 6+(-2)] = [9, 4]")
print(f"   Magnitude: ||3u + 2v|| = {np.linalg.norm(result_d):.2f}")

# Visualize all results
plt.figure(figsize=(10, 10))
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

# Plot original vectors
plt.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1,
           color='red', width=0.005, label='u', alpha=0.7)
plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1,
           color='blue', width=0.005, label='v', alpha=0.7)

# Plot results
results = {
```

```
    'u + v': u_plus_v,
    '2u': two_u,
    'u - v': u_minus_v,
    '3u + 2v': result_d
}

colors = ['green', 'orange', 'purple', 'brown']
for (name, vec), color in zip(results.items(), colors):
    plt.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
               color=color, width=0.005, label=name)
    plt.text(vec[0] + 0.2, vec[1] + 0.2, name, fontsize=10)

plt.xlim(-3, 10)
plt.ylim(-3, 7)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Vector Operations Visualization')
plt.legend()
plt.axis('equal')
plt.show()
```

Example 2: Linear Independence Investigation

Problem: Determine whether the following sets of vectors are linearly independent: a) $\mathbf{v}_1 = [1\ 0]$, $\mathbf{v}_2 = [0\ 1]$ b) $\mathbf{u}_1 = [1\ 2]$, $\mathbf{u}_2 = [2\ 4]$ c) $\mathbf{w}_1 = [1\ 2\ 3]$, $\mathbf{w}_2 = [4\ 5\ 6]$, $\mathbf{w}_3 = [7\ 8\ 9]$

Solution:

```python
def analyze_linear_independence(vectors, names):
    """
    Analyze linear independence of a set of vectors.
    Returns detailed information about the independence.
    """
    # Stack vectors as columns of a matrix
    A = np.column_stack(vectors)

    print(f"\nAnalyzing vectors: {names}")
    print(f"Matrix A with vectors as columns:")
    print(A)

    # Calculate rank
    rank = np.linalg.matrix_rank(A)
    n_vectors = len(vectors)
    dimension = vectors[0].shape[0]

    print(f"\nDimension of space: {dimension}")
    print(f"Number of vectors: {n_vectors}")
    print(f"Rank of matrix: {rank}")

    # Determine independence
    if rank == n_vectors:
        print("Result: Vectors are LINEARLY INDEPENDENT")
        print("Reason: Rank equals number of vectors")
    else:
        print("Result: Vectors are LINEARLY DEPENDENT")
        print(f"Reason: Only {rank} out of {n_vectors} vectors are independent")

    # Additional checks for square matrices
    if n_vectors == dimension:
        det = np.linalg.det(A)
        print(f"\nDeterminant: {det:.6f}")
        if abs(det) > 1e-10:
            print("Non-zero determinant confirms linear independence")
        else:
            print("Zero determinant confirms linear dependence")

    # For dependent vectors, try to express one as combination of others
    if rank < n_vectors and n_vectors == 2:
        # Check if v2 is a multiple of v1
        if not np.allclose(vectors[0], 0):
            # Find scalar c such that v2 = c * v1
            c = vectors[1][0] / vectors[0][0] if vectors[0][0] != 0 else None
            if c is not None and np.allclose(vectors[1], c * vectors[0]):
                print(f"\nDependence relation: {names[1]} = {c:.1f} * {names[0]}")

    return rank == n_vectors
```

```
# Part a: Standard basis vectors
print("="*50)
print("Part a: Standard basis vectors in R^2")
v1 = np.array([1, 0])
v2 = np.array([0, 1])
analyze_linear_independence([v1, v2], ['v1=[1,0]', 'v2=[0,1]'])

# Part b: Parallel vectors
print("\n" + "="*50)
print("Part b: Parallel vectors")
u1 = np.array([1, 2])
u2 = np.array([2, 4])
analyze_linear_independence([u1, u2], ['u1=[1,2]', 'u2=[2,4]'])

# Part c: Three vectors in R^3
print("\n" + "="*50)
print("Part c: Three vectors in R^3")
w1 = np.array([1, 2, 3])
w2 = np.array([4, 5, 6])
w3 = np.array([7, 8, 9])
analyze_linear_independence([w1, w2, w3], ['w1=[1,2,3]', 'w2=[4,5,6]', 'w3=[7,8,9]'])

# Bonus: Show that w3 = -w1 + 2*w2
print("\nVerifying dependence relation:")
combination = -1 * w1 + 2 * w2
print(f"-w1 + 2*w2 = {combination}")
print(f"w3 = {w3}")
print(f"Are they equal? {np.allclose(combination, w3)}")
```

Example 3: Verifying Vector Space Axioms

Problem: Verify that $\mathbb{R}^2$ satisfies the vector space axioms using specific examples.

Solution:

```
def verify_vector_space_axioms():
    """
    Verify vector space axioms for R^2 with concrete examples.
    """
    # Test vectors
    u = np.array([1, 2])
    v = np.array([3, -1])
    w = np.array([-2, 4])
    zero = np.array([0, 0])

    # Scalars
    c = 2
    d = -3

    print("Verifying Vector Space Axioms for R^2")
    print("=====================================")
    print(f"Test vectors: u = {u}, v = {v}, w = {w}")
    print(f"Test scalars: c = {c}, d = {d}\n")

    # Axiom 1: Closure under addition
    print("1. Closure under addition:")
    result = u + v
    print(f"   u + v = {u} + {v} = {result}")
    print(f"   Result is in R^2? {result.shape == (2,)} ✓")

    # Axiom 2: Closure under scalar multiplication
    print("\n2. Closure under scalar multiplication:")
    result = c * v
    print(f"   c * v = {c} * {v} = {result}")
    print(f"   Result is in R^2? {result.shape == (2,)} ✓")

    # Axiom 3: Associativity of addition
    print("\n3. Associativity of addition:")
    left = (u + v) + w
    right = u + (v + w)
    print(f"   (u + v) + w = {u + v} + {w} = {left}")
    print(f"   u + (v + w) = {u} + {v + w} = {right}")
    print(f"   Equal? {np.allclose(left, right)} ✓")

    # Axiom 4: Commutativity of addition
    print("\n4. Commutativity of addition:")
    left = u + v
    right = v + u
    print(f"   u + v = {left}")
    print(f"   v + u = {right}")
    print(f"   Equal? {np.allclose(left, right)} ✓")

    # Axiom 5: Additive identity (zero vector)
```

```
    print("\n5. Additive identity:")
    result = v + zero
    print(f"   v + 0 = {v} + {zero} = {result}")
    print(f"   Equal to v? {np.allclose(result, v)} ✓")

    # Axiom 6: Additive inverse
    print("\n6. Additive inverse:")
    neg_v = -v
    result = v + neg_v
    print(f"   v + (-v) = {v} + {neg_v} = {result}")
    print(f"   Equal to zero? {np.allclose(result, zero)} ✓")

    # Axiom 7: Multiplicative identity
    print("\n7. Multiplicative identity:")
    result = 1 * v
    print(f"   1 * v = 1 * {v} = {result}")
    print(f"   Equal to v? {np.allclose(result, v)} ✓")

    # Axiom 8: Distributivity
    print("\n8. Distributivity properties:")

    # 8a: c(u + v) = cu + cv
    left = c * (u + v)
    right = c * u + c * v
    print(f"   a) c(u + v) = {c} * {u + v} = {left}")
    print(f"      cu + cv = {c * u} + {c * v} = {right}")
    print(f"      Equal? {np.allclose(left, right)} ✓")

    # 8b: (c + d)v = cv + dv
    left = (c + d) * v
    right = c * v + d * v
    print(f"\n   b) (c + d)v = {c + d} * {v} = {left}")
    print(f"      cv + dv = {c * v} + {d * v} = {right}")
    print(f"      Equal? {np.allclose(left, right)} ✓")

    # 8c: c(dv) = (cd)v
    left = c * (d * v)
    right = (c * d) * v
    print(f"\n   c) c(dv) = {c} * {d * v} = {left}")
    print(f"      (cd)v = {c * d} * {v} = {right}")
    print(f"      Equal? {np.allclose(left, right)} ✓")

    print("\n✓ All vector space axioms verified for R^2!")

# Run the verification
verify_vector_space_axioms()
```

Example 4: Building Intuition for Span

Problem: Explore the span of different vector sets in $\mathbb{R}^3$ and visualize the geometric objects they create.

```
def visualize_span_3d():
    """
    Visualize spans of different vector sets in 3D.
    """
    fig = plt.figure(figsize=(15, 5))

    # Case 1: Span of one vector (line)
    ax1 = fig.add_subplot(131, projection='3d')
    v1 = np.array([1, 2, 1])

    # Generate points on the line
    t = np.linspace(-2, 2, 100)
    line = np.array([t * v1[0], t * v1[1], t * v1[2]])

    ax1.plot(line[0], line[1], line[2], 'b-', linewidth=2)
    ax1.quiver(0, 0, 0, v1[0], v1[1], v1[2], color='red', arrow_length_ratio=0.1)
    ax1.set_xlabel('X')
    ax1.set_ylabel('Y')
    ax1.set_zlabel('Z')
    ax1.set_title('Span of 1 Vector: Line')

    # Case 2: Span of two independent vectors (plane)
    ax2 = fig.add_subplot(132, projection='3d')
    v1 = np.array([1, 0, 0])
    v2 = np.array([0, 1, 1])

    # Generate points on the plane
    s = np.linspace(-2, 2, 20)
    t = np.linspace(-2, 2, 20)
    S, T = np.meshgrid(s, t)

    # Plane equation: point = s*v1 + t*v2
    X = S * v1[0] + T * v2[0]
    Y = S * v1[1] + T * v2[1]
    Z = S * v1[2] + T * v2[2]

    ax2.plot_surface(X, Y, Z, alpha=0.3, color='green')
    ax2.quiver(0, 0, 0, v1[0], v1[1], v1[2], color='red', arrow_length_ratio=0.3)
    ax2.quiver(0, 0, 0, v2[0], v2[1], v2[2], color='blue', arrow_length_ratio=0.3)
    ax2.set_xlabel('X')
    ax2.set_ylabel('Y')
    ax2.set_zlabel('Z')
    ax2.set_title('Span of 2 Independent Vectors: Plane')

    # Case 3: Span of three independent vectors (all of R^3)
    ax3 = fig.add_subplot(133, projection='3d')
    v1 = np.array([1, 0, 0])
    v2 = np.array([0, 1, 0])
```

```
    v3 = np.array([0, 0, 1])

    # Show basis vectors and some linear combinations
    ax3.quiver(0, 0, 0, v1[0], v1[1], v1[2], color='red', arrow_length_ratio=0.2, linewidth=3)
    ax3.quiver(0, 0, 0, v2[0], v2[1], v2[2], color='green', arrow_length_ratio=0.2, linewidth=3)
    ax3.quiver(0, 0, 0, v3[0], v3[1], v3[2], color='blue', arrow_length_ratio=0.2, linewidth=3)

    # Show that we can reach any point
    random_points = np.random.uniform(-2, 2, (50, 3))
    ax3.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
                alpha=0.3, s=20, c='purple')

    ax3.set_xlabel('X')
    ax3.set_ylabel('Y')
    ax3.set_zlabel('Z')
    ax3.set_title('Span of 3 Independent Vectors: All of R³')
    ax3.text(1.2, 0, 0, 'e1', fontsize=12)
    ax3.text(0, 1.2, 0, 'e2', fontsize=12)
    ax3.text(0, 0, 1.2, 'e3', fontsize=12)

    plt.tight_layout()
    plt.show()

    # Additional analysis
    print("Span Analysis:")
    print("-" * 40)
    print("1. One vector spans a line through the origin")
    print("   - All points of form: t * v where t ∈ ℝ")
    print("\n2. Two independent vectors span a plane through the origin")
    print("   - All points of form: s * v1 + t * v2 where s, t ∈ ℝ")
    print("\n3. Three independent vectors in R³ span all of R³")
    print("   - Any point can be written as: a * v1 + b * v2 + c * v3")

visualize_span_3d()
```

Common Pitfalls

As you begin your journey with vectors and vector spaces, watch out for these common misconceptions:

1. Confusing Vectors with Points

- **Mistake:** Thinking vectors and points are the same thing
- **Reality:** A vector has magnitude and direction; a point has only position
- **Fix:** Remember that vectors represent displacements, not locations

2. Row vs Column Vector Confusion

- **Mistake:** Using row and column vectors interchangeably
- **Reality:** They have different shapes and behave differently in matrix operations
- **Fix:** Default to column vectors unless specifically working with row vectors

```
# Common mistake
v = np.array([1, 2, 3])  # This creates a 1D array, not a column vector

# Better approach for column vector
v_column = np.array([[1], [2], [3]])  # Shape: (3, 1)
# Or use reshape
v_column = v.reshape(-1, 1)
```

3. Misunderstanding Linear Independence

- **Mistake:** Thinking vectors must be perpendicular to be linearly independent
- **Reality:** Vectors just need to not be expressible as combinations of each other
- **Fix:** Two non-parallel vectors in 2D are always linearly independent

4. Forgetting the Zero Vector Requirement

- **Mistake:** Thinking any line or plane is a subspace
- **Reality:** Only lines/planes through the origin are subspaces
- **Fix:** Always check if the zero vector is included

5. Scalar Multiplication vs Component-wise Multiplication

- **Mistake:** Using `*` for dot product or element-wise multiplication incorrectly
- **Reality:** In NumPy, `*` is element-wise; use `np.dot()` for dot product
- **Fix:** Be explicit about which operation you want

```
# Different multiplication operations
a = np.array([1, 2])
b = np.array([3, 4])

scalar_mult = 5 * a           # [5, 10] - scalar multiplication
element_wise = a * b          # [3, 8] - element-wise multiplication
dot_product = np.dot(a, b)    # 11 - dot product (next lesson!)
```

6. Assuming All Sets of n Vectors in $\mathbb{R}^n$ Are Linearly Independent

- **Mistake:** Thinking that having n vectors in n-dimensional space guarantees independence
- **Reality:** The vectors could still be dependent (e.g., if two are parallel)
- **Fix:** Always verify independence by checking rank or determinant

Summary & Key Takeaways

Today we've built the foundation of linear algebra from the ground up. Here are the essential points to remember:

Core Concepts

- **Scalars** are single numbers; **vectors** are ordered lists of numbers
- Vectors can be interpreted geometrically as arrows with magnitude and direction
- Vector addition follows the parallelogram rule: $(u_1, u_2) + (v_1, v_2) = (u_1 + v_1, u_2 + v_2)$
- Scalar multiplication scales all components: $c(v_1, v_2) = (cv_1, cv_2)$

Linear Independence

- Vectors are **linearly independent** if no vector can be written as a combination of others
- The equation $c_1\mathbf{v}_1 + \ldots + c_k\mathbf{v}_k = \mathbf{0}$ only has the trivial solution
- Maximum number of linearly independent vectors in $\mathbb{R}^n$ is n

Vector Spaces and Subspaces

- A **vector space** satisfies 8 axioms (closure, associativity, identity, etc.)
- **Basis vectors** are linearly independent vectors that span the space
- The **span** of vectors is all possible linear combinations
- **Subspaces** must contain zero and be closed under addition and scalar multiplication

Key Formulas Reference

Vector Magnitude:

$$||\mathbf{v}|| = \sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}$$

Linear Combination:

$$\mathbf{w} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \ldots + c_k\mathbf{v}_k$$

Vector Space Requirements: - Contains zero vector: $\mathbf{0} \in V$ - Closed under addition: $\mathbf{u} + \mathbf{v} \in V$ - Closed under scalar multiplication: $c\mathbf{v} \in V$

NumPy Essentials

```
# Creation
v = np.array([1, 2, 3])

# Operations
u + v                  # Addition
c * v                  # Scalar multiplication
np.linalg.norm(v)      # Magnitude
np.column_stack([u,v]) # Matrix from vectors

# Checking independence
np.linalg.matrix_rank(A)  # Rank of matrix
np.linalg.det(A)         # Determinant (if square)
```

Tomorrow's Preview

In our next episode, we'll explore one of the most important operations in linear algebra: the **dot product**. This operation will unlock:

- **Measuring angles** between vectors
- **Projecting** one vector onto another
- **Cosine similarity** - a fundamental tool in machine learning
- The geometric meaning of perpendicularity

Questions to Ponder

Before tomorrow's lesson, think about these questions:

1. If you have two vectors, how might you define a notion of "how aligned" they are?
2. Given a vector pointing northeast and another pointing east, how would you find the "eastward component" of the first vector?
3. In machine learning, we often want to measure how similar two data points are. How might vectors help with this?

These questions all relate to the dot product, which provides a way to multiply vectors that captures geometric relationships. Tomorrow, we'll see how this simple operation forms the basis for everything from recommendation systems to neural network computations.

See you next time for "Dot Products, Projections, and Cosine Similarity"!

Generated by Archon — AI Learning System